

LeetCode 002: The Printable Problem Set

Generated: 2026-10-04 · **Format:** problem → answer, made for paper (no colour, no links needed) ·

Language: Python 3

How to use this on paper. For each problem, cover the solution with a sheet of paper, read the problem, and write (or say) three things before you uncover the answer:

1. the **family** and the reason for it,
2. the **Contract Sentence** (recursion) or the **invariant** (loops),
3. the first ~5 lines of code.

Then uncover the answer and compare it line by line. Put a tick in the index box only when your version matched on the important lines. Recognising a solution doesn't count as knowing it; reproducing it does.

Why these problems. They're ordered by how much each one is likely to pay off for you right now:

- **Parts 1 to 3 (recursion, DP, backtracking)** come first. They all depend on one idea: *answers flow up as return values, context flows down as parameters*. Every DP solution here starts with a one-line contract, $f(i)$ = the answer for the subproblem starting at i , so you can memorise the *shape* and not only the code.
- **Parts 4 to 7** are the most common interview families, written to model careful bookkeeping: every backward range is commented `first → last`, step, every pointer loop says where the pointer *lands*, and every function's closing step (final return, un-choose, counter decrement, leftover attach) is marked `# tail`.
- **Parts 8 to 9** cover families with little practice so far (binary search, heaps, monotonic stack).
- **Parts 10 to 11** cover the matrix problems and the two design problems (LRU Cache, and the Rate Limiter, which is a real Microsoft interview question).

Every solution in this document was run against test cases (and, where possible, thousands of random cases against a brute-force answer) before it was printed.

Index (tick when you can reproduce it from a blank page)

	#	Problem	Family	Diff
☐	70	Climbing Stairs	1-D DP	E
☐	198	House Robber	1-D DP	M
☐	91	Decode Ways	1-D DP (counting)	M
☐	322	Coin Change	1-D DP (min)	M
☐	139	Word Break	1-D DP (possible?)	M
☐	300	Longest Increasing Subsequence	1-D DP	M
☐	62	Unique Paths	2-D DP	M
☐	1143	Longest Common Subsequence	2-D DP	M
☐	72	Edit Distance	2-D DP	M
☐	78	Subsets	Backtracking	M
☐	46	Permutations	Backtracking	M
☐	39	Combination Sum	Backtracking	M
☐	79	Word Search	Backtracking on a grid	M
☐	104	Maximum Depth of Binary Tree	Tree, answer up	E
☐	543	Diameter of Binary Tree	Tree, answer up + global	E
☐	98	Validate Binary Search Tree	Tree, context down	M

	#	Problem	Family	Diff
☐	102	Binary Tree Level Order Traversal	Tree BFS	M
☐	236	Lowest Common Ancestor	Tree, answer up	M
☐	200	Number of Islands	Grid DFS	M
☐	994	Rotting Oranges	Multi-source BFS	M
☐	207	Course Schedule	Graph cycle / topo sort	M
☐	206	Reverse Linked List	Linked list	E
☐	21	Merge Two Sorted Lists	Linked list	E
☐	19	Remove Nth Node From End	Linked list, fast/slow	M
☐	61	Rotate List	Linked list	M
☐	141	Linked List Cycle	Fast/slow pointers	E
☐	1	Two Sum	Hashing	E
☐	49	Group Anagrams	Hashing	M
☐	238	Product of Array Except Self	Prefix/suffix	M
☐	560	Subarray Sum Equals K	Prefix sum + hashing	M
☐	3	Longest Substring Without Repeating	Sliding window	M
☐	15	3Sum	Sort + two pointers	M
☐	11	Container With Most Water	Two pointers	M
☐	56	Merge Intervals	Intervals	M
☐	704	Binary Search	Binary search	E
☐	33	Search in Rotated Sorted Array	Binary search	M
☐	153	Find Minimum in Rotated Sorted Array	Binary search	M
☐	875	Koko Eating Bananas	Binary search on the answer	M
☐	20	Valid Parentheses	Stack	E
☐	739	Daily Temperatures	Monotonic stack	M
☐	215	Kth Largest Element	Heap	M
☐	347	Top K Frequent Elements	Counting + buckets	M
☐	54	Spiral Matrix	Matrix simulation	M
☐	48	Rotate Image	Matrix transform	M
☐	146	LRU Cache	Design	M
☐	RL	Rate Limiter	Design	M

Part 0: The one-page cheat sheet

Pick the family from what the question asks.

The question says / the input looks like	Family
"list all ...", "return every ..."	Backtracking (carry the path down, collect at the leaf)
" how many ways", " max / min ", " is it possible ", and choices overlap	DP (answer flows up as the return value)
sorted array, or "smallest k such that ..."	Binary search
contiguous subarray / substring	Sliding window, or prefix sums + hash map
pair/triple in a sorted array	Two pointers moving inward

The question says / the input looks like	Family
shortest path / minimum steps / spreads in "minutes"	BFS by levels
connected regions, cycles, prerequisites	DFS / topological sort
"next greater / warmer / taller"	Monotonic stack
"top k", "kth largest"	Heap (or buckets)

Constraints → the complexity you need. $n \leq 20$: 2^n is fine. $n \leq 500$: n^3 . $n \leq 5,000$: n^2 . $n \leq 10^5$ to 10^6 : $n \log n$ or n .

The two styles of recursion.

Style	Parameters carry	Returns	Use for	Memoisable?
Answer-up (the contract)	only <i>where you are</i>	the answer for that subproblem	count / max / min / possible (DP)	Yes
Accumulate-down (the builder)	the path so far	nothing; appends to a result list at the leaf	enumerate all (backtracking)	No

The Contract Sentence (write it before any recursive code). $f(i) = \text{<the answer> for <the subproblem } s[i:]>$. Memo test: *if I call f twice with the same arguments, must it return the same thing?* If not, something that belongs in the return value is sitting in the parameters.

i means "start of what's left". So $i == \text{len}(s)$ means *the remaining problem is empty*, and the base case is the answer for an empty problem:

Asking for	Empty problem answer
number of ways	1 (one way to do nothing)
max / min sum, length	0
fewest coins for amount 0	0
is it possible?	True

Ranges. $\text{range}(\text{first}, \text{last} + \text{step}, \text{step})$. Forward $0 \rightarrow n-1$: $\text{range}(0, n)$. Backward $n-1 \rightarrow 0$: $\text{range}(n-1, -1, -1)$. Backward bottom $\rightarrow$ top: $\text{range}(\text{bottom}, \text{top} - 1, -1)$. $\text{range}(2, 2)$ is empty.

Closing checklist (30 seconds, before tracing).

1. Every path out of every function returns something (check the very bottom).
2. Every choose has an un-choose.
3. Every computed result is stored (memo, visited, answer).
4. Every counter that should shrink, shrinks.
5. The code does what the comment above it says.

Trace before Submit. Smallest breaking input first (empty, one element, $1 \times n$). One row per loop pass or call. Write every range out as a literal list.

Python for a C programmer.

C intuition	Python reality
strlen is $O(n)$	$\text{len}()$ is $O(1)$
a mutating call returns the object	<code>.append</code> <code>.sort</code> <code>.reverse</code> <code>.extend</code> return None; never chain or assign them
booleans are 0/1	True, False
deep recursion is fine	default limit is about 1,000 calls; go bottom-up or iterative for deep inputs
$[[0]*n]*m$ makes a grid	it aliases one row m times; use $[[0]*n \text{ for } _ \text{ in range}(m)]$
out-of-range slice crashes	$s[i:j]$ is half-open and never raises; $s[\text{len}(s):] == ""$

Toolbox: collections.deque (append, appendleft, popleft, len(q)), defaultdict(list), Counter, heapq (min-heap: heappush, heappop, heap[0]), bisect_left, functools.cache, enumerate, zip, divmod.

Part 1: 1-D dynamic programming (answer flows up)

70. Climbing Stairs (Easy)

Problem. You are at step 0 and want to reach step n . Each move climbs 1 or 2 steps. How many distinct ways are there to reach step n ?

Example. $n = 3 \rightarrow 3$ (1+1+1, 1+2, 2+1). Constraint: $1 \leq n \leq 45$.

Contract. $f(i)$ = number of ways to get from step i to step n .

```
from functools import cache

def climbStairs(n):
    @cache
    def f(i):
        if i == n: return 1          # nothing left to climb: exactly one way
        if i > n: return 0           # overshoot: no ways
        return f(i + 1) + f(i + 2)   # take 1 step, or take 2 steps
    return f(0)
```

Bottom-up (the same recurrence, keeping only the two answers you need):

```
def climbStairs(n):
    one_ahead, two_ahead = 1, 0      # f(n) = 1, f(n+1) = 0
    for i in range(n - 1, -1, -1):   # n-1  $\rightarrow$  0, step -1
        one_ahead, two_ahead = one_ahead + two_ahead, one_ahead
    return one_ahead                 # f(0)
```

$O(n)$ time. $O(1)$ space bottom-up.

198. House Robber (Medium)

Problem. `nums[i]` is the money in house `i`. You can't rob two adjacent houses. Return the maximum money you can rob.

Example. `[2,7,9,3,1] → 12 (2 + 9 + 1)`. `[1,2,3,1] → 4`.

Contract. `f(i)` = max money from houses `nums[i:]`. (The running total is **not** a parameter. It's what `f` returns.)

```
from functools import cache

def rob(nums):
    n = len(nums)
    @cache
    def f(i):
        if i >= n: return 0                # no houses left: 0
        take = nums[i] + f(i + 2)          # rob i, so skip i+1
        skip = f(i + 1)                    # don't rob i
        return max(take, skip)
    return f(0)
```

Bottom-up:

```
def rob(nums):
    next1, next2 = 0, 0                    # f(i+1), f(i+2)
    for i in range(len(nums) - 1, -1, -1): # n-1 → 0, step -1
        cur = max(nums[i] + next2, next1)
        next2, next1 = next1, cur
    return next1                           # f(0)
```

$O(n)$ time.

91. Decode Ways (Medium)

Problem. Letters map to numbers: A = 1, ..., Z = 26. Given a digit string `s`, return how many ways it can be decoded. "06" isn't a valid code for F (no leading zeros).

Example. "12" → 2 (AB, L). "226" → 3. "06" → 0. "10" → 1.

Contract. `f(i)` = number of ways to decode the suffix `s[i:]`.

```
from functools import cache

def numDecodings(s):
    n = len(s)
    @cache
    def f(i):
        if i == n: return 1                # empty suffix: one way (decode nothing)
        if s[i] == '0': return 0           # no letter starts with 0
        ways = f(i + 1)                     # use one digit
        if i + 1 < n and 10 <= int(s[i:i + 2]) <= 26:
            ways += f(i + 2)                # use two digits
        return ways
    return f(0)
```

$O(n)$ time. This is "how many", so it's DP, not backtracking.

322. Coin Change (Medium)

Problem. Given coin values `coins` (unlimited supply of each) and an amount, return the fewest coins that sum to amount, or -1 if it's impossible.

Example. `coins = [1,2,5]`, `amount = 11` → 3 (5+5+1). `coins = [2]`, `amount = 3` → -1. `amount = 0` → 0.
Constraint: `amount ≤ 104`.

Contract. `f(a)` = fewest coins that make amount `a` (infinity if impossible).

```
from functools import cache

def coinChange(coins, amount):
    INF = float('inf')
    @cache
    def f(a):
        if a == 0: return 0                # nothing left to make: 0 coins
        best = INF
        for c in coins:
            if c <= a:
                best = min(best, 1 + f(a - c)) # use coin c, solve the rest
        return best
    ans = f(amount)
    return -1 if ans == INF else ans
```

The top-down version can recurse 10^4 deep, past Python's limit. In an interview, write the bottom-up table:

```
def coinChange(coins, amount):
    INF = amount + 1                # more coins than could ever be needed
    dp = [0] + [INF] * amount        # dp[a] = fewest coins for amount a
    for a in range(1, amount + 1):   # 1 → amount
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] <= amount else -1
```

$O(\text{amount} \times \text{len}(\text{coins}))$ time.

139. Word Break (Medium)

Problem. Given a string s and a list of words, return True if s can be split into a sequence of one or more dictionary words (words may be reused).

Example. $s = \text{"leetcode"}, [\text{"leet"}, \text{"code"}] \rightarrow \text{True}$. $s = \text{"catsanddog"}, [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}] \rightarrow \text{False}$.

Contract. $f(i)$ = can the suffix $s[i:]$ be split into words?

```
from functools import cache

def wordBreak(s, wordDict):
    words = set(wordDict)
    n = len(s)
    @cache
    def f(i):
        if i == n: return True          # empty suffix: trivially splittable
        for j in range(i + 1, n + 1):  # i+1 → n (j is the exclusive end)
            if s[i:j] in words and f(j):
                return True
        return False                    # tail: no split worked
    return f(0)
```

$O(n^2)$ calls to in on slices (about n^3 characters). Fine for $n \leq 300$.

300. Longest Increasing Subsequence (Medium)

Problem. Return the length of the longest strictly increasing subsequence of nums (a subsequence keeps order but may skip elements).

Example. $[10, 9, 2, 5, 3, 7, 101, 18] \rightarrow 4$ (2, 3, 7, 101). $[7, 7, 7] \rightarrow 1$.

Contract. $\text{dp}[i]$ = length of the longest increasing subsequence that **starts at** i .

```
def lengthOfLIS(nums):
    n = len(nums)
    dp = [1] * n          # each element alone has length 1
    for i in range(n - 1, -1, -1):  # n-1 → 0, step -1
        for j in range(i + 1, n):  # i+1 → n-1
            if nums[j] > nums[i]:
                dp[i] = max(dp[i], 1 + dp[j])
    return max(dp)
```

$O(n^2)$. The $O(n \log n)$ follow-up (memorise it as a pattern):

```
from bisect import bisect_left

def lengthOfLIS(nums):
    tails = []          # tails[k] = smallest tail of any increasing subsequence of length k+1
    for x in nums:
        k = bisect_left(tails, x)
        if k == len(tails):
            tails.append(x)          # x extends the longest one
        else:
            tails[k] = x             # x is a smaller tail for length k+1
    return len(tails)
```

Part 2: 2-D dynamic programming

62. Unique Paths (Medium)

Problem. A robot starts at the top-left of an $m \times n$ grid and can only move right or down. How many paths reach the bottom-right?

Example. $m = 3, n = 7 \rightarrow 28$. $m = 3, n = 2 \rightarrow 3$.

Contract. $f(r, c)$ = number of paths from (r, c) to $(m-1, n-1)$.

```
from functools import cache

def uniquePaths(m, n):
    @cache
    def f(r, c):
        if r == m - 1 and c == n - 1: return 1    # already there: one path
        if r >= m or c >= n: return 0             # fell off the grid
        return f(r + 1, c) + f(r, c + 1)          # down, or right
    return f(0, 0)
```

$O(m \times n)$.

1143. Longest Common Subsequence (Medium)

Problem. Return the length of the longest subsequence that appears in both text1 and text2.

Example. "abcde", "ace" $\rightarrow 3$. "abc", "def" $\rightarrow 0$. (1035 Uncrossed Lines is this exact problem on integer arrays.)

Contract. $f(i, j)$ = LCS length of text1[i:] and text2[j:]. Three things are involved: i , j , and the answer. The answer is the **return value**, not a third parameter.

```
from functools import cache

def longestCommonSubsequence(text1, text2):
    @cache
    def f(i, j):
        if i == len(text1) or j == len(text2):
            return 0                                # one suffix is empty
        if text1[i] == text2[j]:
            return 1 + f(i + 1, j + 1)              # match: use both
        return max(f(i + 1, j), f(i, j + 1))        # drop one or the other
    return f(0, 0)
```

Bottom-up ($dp[i][j]$ is $f(i, j)$; the extra row m and column n are the empty-suffix zeros):

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m - 1, -1, -1):                # m-1  $\rightarrow$  0, step -1
        for j in range(n - 1, -1, -1):            # n-1  $\rightarrow$  0, step -1
            if text1[i] == text2[j]:
                dp[i][j] = 1 + dp[i + 1][j + 1]
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
    return dp[0][0]
```

$O(m \times n)$. Use bottom-up when the strings can be 1,000 long (recursion depth).

72. Edit Distance (Medium)

Problem. Return the minimum number of single-character operations (insert, delete, replace) to turn word1 into word2.

Example. "horse", "ros" → 3. "intention", "execution" → 5.

Contract. $f(i, j) = \text{min edits to turn word1[i:] into word2[j:]}$.

```
from functools import cache

def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    @cache
    def f(i, j):
        if i == m: return n - j          # insert the rest of word2
        if j == n: return m - i          # delete the rest of word1
        if word1[i] == word2[j]:
            return f(i + 1, j + 1)        # free: characters already match
        return 1 + min(f(i + 1, j),      # delete word1[i]
                       f(i, j + 1),      # insert word2[j]
                       f(i + 1, j + 1))  # replace word1[i] with word2[j]
    return f(0, 0)
```

$O(m \times n)$.

Part 3: Backtracking (carry the path down, choose / explore / un-choose)

78. Subsets (Medium)

Problem. Given distinct integers nums, return all possible subsets (any order).

Example. [1,2,3] → [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]].

```
def subsets(nums):
    res, path = [], []
    def bt(i):
        # decide nums[i:]
        if i == len(nums):
            res.append(path[:])
            return
        path.append(nums[i])
        bt(i + 1)
        path.pop()
        bt(i + 1)
        # choose: include nums[i]
        # explore
        # un-choose
        # exclude nums[i]
    bt(0)
    return res
```

$O(n \times 2^n)$. "All" → backtracking.

46. Permutations (Medium)

Problem. Given distinct integers `nums`, return all orderings.

Example. `[1,2,3] → 6` permutations.

```
def permute(nums):
    res, path = [], []
    used = [False] * len(nums)
    def bt():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for k in range(len(nums)):
            if used[k]:
                continue
            used[k] = True          # choose (two things)
            path.append(nums[k])
            bt()                    # explore
            path.pop()              # un-choose (both things)
            used[k] = False
    bt()
    return res
```

$O(n \times n!)$.

39. Combination Sum (Medium)

Problem. Given distinct positive candidates and a target, return all unique combinations that sum to target. A number may be used any number of times.

Example. `[2,3,6,7], 7 → [[2,2,3],[7]]`.

```
def combinationSum(candidates, target):
    res, path = [], []
    def bt(start, remaining):
        if remaining == 0:
            res.append(path[:])
            return
        for k in range(start, len(candidates)):
            c = candidates[k]
            if c > remaining:
                continue
            path.append(c)
            bt(k, remaining - c)      # k, not k+1: c may be reused
            path.pop()
    bt(0, target)
    return res
```

Carrying remaining down is fine here: we're **listing** solutions, not computing one answer. Starting at start stops `[2,3]` and `[3,2]` both appearing.

79. Word Search (Medium)

Problem. Given a grid of letters and a word, return True if the word can be traced through horizontally or vertically adjacent cells, using each cell at most once.

Example. board `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`: "ABCCED" → True, "ABCB" → False.

Contract. `dfs(r, c, k) = can word[k:] be matched starting at cell (r, c)?` (Guard style: call freely, reject bad cells at entry.)

```
def exist(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, k):
        if k == len(word): return True                # nothing left to match
        if r < 0 or r >= R or c < 0 or c >= C or board[r][c] != word[k]:
            return False
        saved = board[r][c]
        board[r][c] = '#'                             # choose: mark as used
        found = (dfs(r + 1, c, k + 1) or dfs(r - 1, c, k + 1) or
                 dfs(r, c + 1, k + 1) or dfs(r, c - 1, k + 1))
        board[r][c] = saved                           # un-choose, ALWAYS
        return found
    for r in range(R):
        for c in range(C):
            if dfs(r, c, 0):
                return True
    return False                                     # tail
```

$O(R \times C \times 3^{\text{len}(\text{word})})$.

Part 4: Trees (context down, answer up)

All tree problems use:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

104. Maximum Depth of Binary Tree (Easy)

Problem. Return the number of nodes on the longest path from the root down to a leaf.

Example. `[3,9,20,null,null,15,7]` → 3. Empty tree → 0.

Contract. `maxDepth(node) = depth of the subtree rooted at node.`

```
def maxDepth(root):
    if not root: return 0                # empty tree: depth 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

543. Diameter of Binary Tree (Easy)

Problem. Return the length (in edges) of the longest path between any two nodes. The path may not pass through the root.

Example. [1,2,3,4,5] → 3 (4 → 2 → 1 → 3).

Contract. height(node) returns what the **parent** needs (height). It also *records* what the **problem** asks (best path through this node).

```
def diameterOfBinaryTree(root):
    best = 0
    def height(node):
        nonlocal best
        if not node: return 0
        L = height(node.left)
        R = height(node.right)
        best = max(best, L + R)          # longest path bending at node
        return 1 + max(L, R)           # parent can only use one side
    height(root)
    return best
```

98. Validate Binary Search Tree (Medium)

Problem. Return True if the tree is a valid BST: every node in a left subtree is strictly less than the node, and every node in a right subtree is strictly greater.

Example. [2,1,3] → True. [5,1,4,null,null,3,6] → False. Trap: [5,4,6,null,null,3,7] → False (3 is in 5's right subtree).

```
def isValidBST(root):
    def ok(node, lo, hi):                # context down: allowed open range (lo, hi)
        if not node: return True
        if not (lo < node.val < hi): return False
        return ok(node.left, lo, node.val) and ok(node.right, node.val, hi)
    return ok(root, float('-inf'), float('inf'))
```

Comparing each node only with its children is the classic wrong answer. The range has to be passed down.

102. Binary Tree Level Order Traversal (Medium)

Problem. Return the node values level by level, left to right.

Example. [3,9,20,null,null,15,7] → [[3],[9,20],[15,7]].

```
from collections import deque

def levelOrder(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):          # exactly the nodes of this level
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

The for `_ in range(len(q))` level loop is the same trick as Rotting Oranges.

236. Lowest Common Ancestor of a Binary Tree (Medium)

Problem. Given two nodes p and q in a binary tree, return their lowest common ancestor (a node may be its own ancestor).

Example. In [3,5,1,6,2,0,8,null,null,7,4]: LCA(5, 1) = 3, LCA(5, 4) = 5.

Contract. Returns p or q if exactly one is in this subtree, the LCA if both are, None if neither is.

```
def lowestCommonAncestor(root, p, q):
    if not root or root is p or root is q:
        return root
    L = lowestCommonAncestor(root.left, p, q)
    R = lowestCommonAncestor(root.right, p, q)
    if L and R: return root          # one on each side: root is the split
    return L or R                   # pass up whichever side found something
```

Part 5: Grids and graphs

200. Number of Islands (Medium)

Problem. In a grid of '1' (land) and '0' (water), count the islands (groups of land connected horizontally or vertically).

Example. `[["1","1","0"],["1","0","0"],["0","0","1"]]` → 2.

```
def numIslands(grid):
    R, C = len(grid), len(grid[0])
    DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))
    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] != '1':
                continue
            count += 1
            grid[r][c] = '0'
            stack = [(r, c)]
            while stack:
                cr, cc = stack.pop()
                for dr, dc in DIRS:
                    nr, nc = cr + dr, cc + dc
                    if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == '1':
                        grid[nr][nc] = '0'
                        stack.append((nr, nc))
    return count
```

$O(R \times C)$. An explicit stack avoids hitting the recursion limit on a 300×300 all-land grid.

994. Rotting Oranges (Medium)

Problem. Grid cells: 0 empty, 1 fresh, 2 rotten. Every minute, fresh oranges next to a rotten one (4 directions) rot. Return the minutes until no fresh orange is left, or -1 if that's impossible.

Example. $[[2,1,1],[1,1,0],[0,1,1]] \rightarrow 4$. $[[2,1,1],[0,1,1],[1,0,1]] \rightarrow -1$. $[[0,2]] \rightarrow 0$.

```
from collections import deque

def orangesRotting(grid):
    R, C = len(grid), len(grid[0])
    DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))
    q, fresh = deque(), 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 2: q.append((r, c)) # every rotten orange starts the BFS
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    while q and fresh > 0:
        for _ in range(len(q)): # one minute = one BFS level
            r, c = q.popleft()
            for dr, dc in DIRS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1 # the counter must shrink
                    q.append((nr, nc))
            minutes += 1
    return minutes if fresh == 0 else -1 # tail
```

$O(R \times C)$.

207. Course Schedule (Medium)

Problem. There are numCourses courses. [a, b] in prerequisites means you must take b before a. Return True if you can finish every course (that is, there's no cycle).

Example. 2, [[1,0]] → True. 2, [[1,0],[0,1]] → False.

DFS with three states (0 unvisited, 1 on the current path, 2 finished):

```
def canFinish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    for a, b in prerequisites:
        graph[b].append(a)          # edge b → a
    state = [0] * numCourses
    def has_cycle(u):
        if state[u] == 1: return True    # back on the current path: cycle
        if state[u] == 2: return False   # already proven safe
        state[u] = 1
        for v in graph[u]:
            if has_cycle(v):
                return True
        state[u] = 2
        return False                  # tail
    for u in range(numCourses):
        if has_cycle(u):
            return False
    return True                      # tail: don't forget this one
```

Kahn's algorithm (BFS, no recursion, and it also gives you an order for 210 Course Schedule II):

```
from collections import deque

def canFinish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses        # number of unmet prerequisites
    for a, b in prerequisites:
        graph[b].append(a)
        indeg[a] += 1
    q = deque(u for u in range(numCourses) if indeg[u] == 0)
    taken = 0
    while q:
        u = q.popleft()
        taken += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return taken == numCourses      # any course left over is in a cycle
```

$O(V + E)$.

Part 6: Linked lists (ask "where does the pointer land?")

All linked-list problems use:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
```

206. Reverse Linked List (Easy)

Problem. Reverse a singly linked list and return the new head.

Example. 1→2→3→4→5 → 5→4→3→2→1.

```
def reverseList(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next          # 1. save
        cur.next = prev         # 2. flip
        prev = cur              # 3. advance prev
        cur = nxt               # 4. advance cur
    return prev                 # landing: cur is None, prev is the new head
```

O(n) time, O(1) space. Write this from memory until it's automatic. Reorder List, Palindrome List and K-Group Reverse all use it.

21. Merge Two Sorted Lists (Easy)

Problem. Merge two sorted linked lists into one sorted list.

Example. 1→2→4 and 1→3→4 → 1→1→2→3→4→4.

```
def mergeTwoLists(l1, l2):
    dummy = tail = ListNode()          # dummy head: no special case for the first node
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2                # tail: attach whatever is left
    return dummy.next
```

19. Remove Nth Node From End of List (Medium)

Problem. Remove the n-th node from the end of the list and return the head.

Example. 1→2→3→4→5, n = 2 → 1→2→3→5. [1], n = 1 → [].

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)          # handles removing the head itself
    fast = slow = dummy
    for _ in range(n):                  # gap: fast is n nodes ahead of slow
        fast = fast.next
    while fast.next:                    # landing: fast on the LAST node
        fast, slow = fast.next, slow.next # so slow is just BEFORE the target
    slow.next = slow.next.next
    return dummy.next
```

One pass, O(1) space.

61. Rotate List (Medium)

Problem. Rotate a linked list to the right by k places.

Example. 1→2→3→4→5, k = 2 → 4→5→1→2→3. 0→1→2, k = 4 → 2→0→1.

```
def rotateRight(head, k):
    if not head or not head.next:
        return head
    n, tail = 1, head
    while tail.next:                  # landing: tail on the last node, n = length
        tail = tail.next
        n += 1
    k %= n
    if k == 0:
        return head
    new_tail = head
    for _ in range(n - k - 1):        # walk to index n-k-1: the new tail
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None              # cut
    tail.next = head                  # reconnect the old tail to the old head
    return new_head
```

Trace on 5 nodes, k = 2: n - k - 1 = 2 steps, so new_tail lands on node 3 and new_head is node 4. Edge cases: [], one node, k % n == 0.

141. Linked List Cycle (Easy)

Problem. Return True if the linked list has a cycle.

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:          # fast needs two valid steps
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

O(n) time, O(1) space (Floyd's tortoise and hare).

Part 7: Arrays, hashing, two pointers, sliding window

1. Two Sum (Easy)

Problem. Return the indices of the two numbers that add up to target (exactly one answer exists; don't use the same element twice).

Example. [2,7,11,15], 9 → [0,1]. [3,3], 6 → [0,1].

```
def twoSum(nums, target):
    seen = {}                      # value → index
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i                # store AFTER checking (x can't pair with itself)
```

O(n).

49. Group Anagrams (Medium)

Problem. Group the strings that are anagrams of each other.

Example. ["eat", "tea", "tan", "ate", "nat", "bat"] → [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]] (any order).

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s))      # sorted() returns a list: join it into a str key
        groups[key].append(s)
    return list(groups.values())
```

O(n × k log k). Alternative key: tuple of 26 letter counts.

238. Product of Array Except Self (Medium)

Problem. Return ans where $\text{ans}[i]$ is the product of every element except $\text{nums}[i]$. No division; $O(n)$.

Example. $[1,2,3,4] \rightarrow [24,12,8,6]$. $[-1,1,0,-3,3] \rightarrow [0,0,9,0,0]$.

```
def productExceptSelf(nums):
    n = len(nums)
    ans = [1] * n
    left = 1
    for i in range(n):                # 0 → n-1
        ans[i] = left                 # product of everything LEFT of i
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):    # n-1 → 0, step -1
        ans[i] *= right               # times everything RIGHT of i
        right *= nums[i]
    return ans
```

$O(n)$ time, $O(1)$ extra space.

560. Subarray Sum Equals K (Medium)

Problem. Return how many contiguous subarrays sum to k . Numbers can be negative (so a sliding window doesn't work).

Example. $[1,1,1], 2 \rightarrow 2$. $[1,2,3], 3 \rightarrow 2$.

```
def subarraySum(nums, k):
    count, prefix = 0, 0
    seen = {0: 1}                    # prefix sum → times seen; the empty prefix is 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)  # earlier prefixes P with prefix - P == k
        seen[prefix] = seen.get(prefix, 0) + 1
    return count
```

$O(n)$. The same "remainder / complement in a hash map" idea solves Divisible Sum Pairs in $O(n + k)$.

3. Longest Substring Without Repeating Characters (Medium)

Problem. Return the length of the longest substring with no repeated characters.

Example. "abcabcbb" → 3. "bbbbb" → 1. "pwwkew" → 3. "" → 0.

Invariant. $s[\text{left} : \text{right}+1]$ has no repeats.

```
def lengthOfLongestSubstring(s):
    last = {}                        # char → last index seen
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:  # duplicate is INSIDE the window
            left = last[ch] + 1              # jump past it (left only moves forward)
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

$O(n)$. Trap: "abba". Without the $\geq$ left check, left would jump backwards.

15. 3Sum (Medium)

Problem. Return all unique triplets [a, b, c] from nums with $a + b + c == 0$.

Example. [-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]]. [0,0,0,0] → [[0,0,0]].

```
def threeSum(nums):
    nums.sort()
    res, n = [], len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue                # skip duplicate anchors
        lo, hi = i + 1, n - 1       # invariant: search nums[i+1 .. n-1]
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0:
                lo += 1              # need bigger
            elif s > 0:
                hi -= 1              # need smaller
            else:
                res.append([nums[i], nums[lo], nums[hi]])
                lo += 1
                hi -= 1              # both move inward
                while lo < hi and nums[lo] == nums[lo - 1]:
                    lo += 1          # skip duplicate second values
    return res
```

$O(n^2)$. Init comes from the invariant: $lo = i + 1$, never 1.

11. Container With Most Water (Medium)

Problem. $height[i]$ is a vertical line at $x = i$. Pick two lines that, with the x-axis, hold the most water. Return that area.

Example. [1,8,6,2,5,4,8,3,7] → 49.

```
def maxArea(height):
    lo, hi, best = 0, len(height) - 1, 0
    while lo < hi:
        best = max(best, (hi - lo) * min(height[lo], height[hi]))
        if height[lo] < height[hi]:
            lo += 1                # the shorter wall limits every narrower container: drop it
        else:
            hi -= 1
    return best
```

$O(n)$.

56. Merge Intervals (Medium)

Problem. Merge all overlapping intervals.

Example. $[[1,3],[2,6],[8,10],[15,18]] \rightarrow [[1,6],[8,10],[15,18]]$. $[[1,4],[2,3]] \rightarrow [[1,4]]$.

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    res = []
    for start, end in intervals:
        if res and start <= res[-1][1]:
            res[-1][1] = max(res[-1][1], end)  # overlaps the last merged one
            # max(): it might be nested inside
        else:
            res.append([start, end])
    return res
```

$O(n \log n)$.

Part 8: Binary search

704. Binary Search (Easy)

Problem. Return the index of target in sorted nums, or -1.

Example. $[-1,0,3,5,9,12], 9 \rightarrow 4$. $[-1,0,3,5,9,12], 2 \rightarrow -1$.

Invariant (closed). If target is present, it's in $\text{nums}[\text{lo}..\text{hi}]$.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

$O(\log n)$.

33. Search in Rotated Sorted Array (Medium)

Problem. A sorted array of distinct values was rotated at an unknown pivot (e.g. [4,5,6,7,0,1,2]). Return the index of target, or -1, in $O(\log n)$.

Example. [4,5,6,7,0,1,2], 0 → 4. [4,5,6,7,0,1,2], 3 → -1.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:           # left half nums[lo..mid] is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                             # right half nums[mid..hi] is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

One half is always sorted. Check whether the target is inside that half.

153. Find Minimum in Rotated Sorted Array (Medium)

Problem. Return the minimum of a rotated sorted array of distinct values, in $O(\log n)$.

Example. [3,4,5,1,2] → 1. [11,13,15,17] → 11.

```
def findMin(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:                       # stop when one candidate is left
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1                 # min is strictly right of mid
        else:
            hi = mid                     # mid itself could be the min: keep it
    return nums[lo]
```

Note the two templates: $lo \leq hi$ with $mid \pm 1$ (find an exact value), and $lo < hi$ with $hi = mid$ (shrink to a boundary).

875. Koko Eating Bananas (Medium)

Problem. Piles of bananas piles, h hours. Each hour Koko picks one pile and eats up to k bananas from it. Return the minimum integer speed k that lets her finish within h hours.

Example. [3,6,7,11], h = 8 → 4. [30,11,23,4,20], h = 5 → 30.

Binary search on the answer. If speed k works, every faster speed works too, so the answers look like no no no YES YES YES. Find the first YES.

```
def minEatingSpeed(piles, h):
    lo, hi = 1, max(piles)          # the answer is in [lo, hi]
    while lo < hi:
        k = (lo + hi) // 2
        hours = sum((p + k - 1) // k for p in piles)    # ceil(p / k) per pile
        if hours <= h:
            hi = k                    # k works: answer is k or smaller
        else:
            lo = k + 1                # k too slow
    return lo
```

O(n log max).

Part 9: Stacks and heaps

20. Valid Parentheses (Easy)

Problem. Given a string of ()[]{}, return True if every bracket is closed by the same type in the right order.

Example. "()[]{}" → True. "()" → False. "[" → False.

```
def isValid(s):
    match = {'(': ')', '[': ']', '{': '}', '(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in match:                # closing bracket
            if not stack or stack.pop() != match[ch]:
                return False
        else:
            stack.append(ch)
    return not stack                    # tail: leftovers are unclosed
```

739. Daily Temperatures (Medium)

Problem. For each day, return how many days until a warmer temperature (0 if never).

Example. [73,74,75,71,69,72,76,73] → [1,1,4,2,1,1,0,0].

```
def dailyTemperatures(temperatures):
    ans = [0] * len(temperatures)
    stack = [] # indices still waiting for a warmer day (temps decreasing)
    for i, t in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < t:
            j = stack.pop()
            ans[j] = i - j # day i is j's first warmer day
        stack.append(i)
    return ans
```

O(n): each index is pushed once and popped at most once.

215. Kth Largest Element in an Array (Medium)

Problem. Return the k-th largest element (in sorted order, not the k-th distinct).

Example. [3,2,1,5,6,4], k = 2 → 5. [3,2,3,1,2,4,5,5,6], k = 4 → 4.

```
import heapq

def findKthLargest(nums, k):
    heap = [] # min-heap holding the k largest seen so far
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap) # drop the smallest of them
    return heap[0] # smallest of the k largest = kth largest
```

O(n log k).

347. Top K Frequent Elements (Medium)

Problem. Return the k most frequent elements (any order; the answer is unique).

Example. [1,1,1,2,2,3], k = 2 → [1,2].

```
from collections import Counter

def topKFrequent(nums, k):
    count = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)] # buckets[f] = values seen f times
    for x, f in count.items():
        buckets[f].append(x)
    res = []
    for f in range(len(nums), 0, -1): # n → 1, step -1
        for x in buckets[f]:
            res.append(x)
            if len(res) == k:
                return res
```

O(n). The heap one-liner is also fine to say out loud: `heapq.nlargest(k, count, key=count.get)`, O(n log k).

Part 10: Matrix

54. Spiral Matrix (Medium)

Problem. Return all elements of an $m \times n$ matrix in clockwise spiral order.

Example. $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow [1,2,3,6,9,8,7,4,5]$. $[[1,2,3,4],[5,6,7,8],[9,10,11,12]] \rightarrow [1,2,3,4,8,12,11,10,9,5,6,7]$.

Invariant (closed). Rows $top..bottom$ and columns $left..right$ are still unvisited.

```
def spiralOrder(matrix):
    res = []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1):           # top row: left → right, +1
            res.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1):           # right col: top → bottom, +1
            res.append(matrix[r][right])
        right -= 1
        if top <= bottom:                           # a row is still left
            for c in range(right, left - 1, -1):    # bottom row: right → left, -1
                res.append(matrix[bottom][c])
            bottom -= 1
        if left <= right:                           # a column is still left
            for r in range(bottom, top - 1, -1):    # left col: bottom → top, -1
                res.append(matrix[r][left])
            left += 1
    return res
```

Every range is `range(first, last + step, step)`. Trace a 1×3 and a 3×1 before trusting it.

48. Rotate Image (Medium)

Problem. Rotate an $n \times n$ matrix 90° clockwise **in place**.

Example. $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow [[7,4,1],[8,5,2],[9,6,3]]$.

```
def rotate(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):                 # upper triangle only, or you swap twice
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c] # transpose
    for row in matrix:
        row.reverse()                             # mirror left ↔ right (returns None)
```

Clockwise = transpose, then reverse each row. Counter-clockwise = transpose, then reverse the row order.

Part 11: Design

146. LRU Cache (Medium)

Problem. Design a cache with capacity `capacity`. `get(key)` returns the value or `-1`. `put(key, value)` inserts or updates; when over capacity, evict the least recently used key. Both must be $O(1)$.

Example. cap 2: `put(1,1)` `put(2,2)` `get(1)→1` `put(3,3)` (evicts 2) `get(2)→-1` `put(4,4)` (evicts 1) `get(1)→-1` `get(3)→3` `get(4)→4`.

Design. Hash map `key → node` plus a doubly linked list ordered by recency, with sentinel head and tail. Two helpers, so the pointer surgery is written exactly once.

```
class Node:
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}                    # key → Node
        self.head, self.tail = Node(), Node()  # head.next = most recent
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _add_front(self, node):
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._add_front(node)          # now most recent
        return node.val

    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node
        self._add_front(node)
        if len(self.map) > self.cap:
            lru = self.tail.prev        # least recent
            self._remove(lru)
            del self.map[lru.key]       # this is why Node stores its key
```

RL. Rate Limiter (Microsoft interview question)

Problem. Allow at most limit requests per user in any sliding window of window seconds. allow(user, t) returns True and records the request if it's allowed, False otherwise. Timestamps arrive in non-decreasing order. Follow-up: don't keep memory for users who have gone quiet.

Example. limit = 2, window = 10: allow("a",1)→True, allow("a",2)→True, allow("a",3)→False, allow("b",3)→True, allow("a",11)→True (the request at t=1 has expired: the window is (t - 10, t]).

Version 1: per-user log with lazy cleanup.

```
from collections import deque, defaultdict

class RateLimiter:
    def __init__(self, limit, window):
        self.limit, self.window = limit, window
        self.log = defaultdict(deque)          # user → accepted timestamps, oldest left

    def allow(self, user, t):
        q = self.log[user]
        while q and q[0] <= t - self.window:   # expire anything outside (t-window, t]
            q.popleft()
        if len(q) < self.limit:
            q.append(t)
            return True
        return False
```

Version 2: one global time-ordered queue as the **single source of truth**, plus a derived per-user count. Expiring only touches requests that have actually expired, and idle users get deleted.

```
from collections import deque

class RateLimiterGlobal:
    def __init__(self, limit, window):
        self.limit, self.window = limit, window
        self.events = deque()                  # (t, user) per accepted request, time-ordered
        self.count = {}                       # user → accepted requests in window (derived)

    def _expire(self, now):
        while self.events and self.events[0][0] <= now - self.window:
            _, user = self.events.popleft()
            self.count[user] -= 1
            if self.count[user] == 0:
                del self.count[user]           # memory: forget idle users

    def allow(self, user, t):
        self._expire(t)
        if self.count.get(user, 0) < self.limit:
            self.events.append((t, user))      # append keeps time order (never re-insert)
            self.count[user] = self.count.get(user, 0) + 1
            return True
        return False
```

Each request is enqueued once and dequeued once, so both versions are **O(1) amortised** per call. Things to say out loud in the interview: the invariant ("events is always sorted by time"), why there's only one owner of the data, and the trade-off against a fixed-window counter (less memory, but it allows bursts at window edges) or a token bucket (O(1) memory per user, smooth rate).